

Web Application Deployment in Azure

This document outlines three methods for deploying a web application to Azure App Services:

- **Option 1: Publishing web files to Azure App Service**
- **Option 2: Deploying via containerization using Docker Hub**
- **Option 3: Deploying via CI/CD pipeline using GitHub Actions**

Each method is tailored for specific scenarios to streamline deployment and management.

Option 1: Publishing Web Files to Azure App Service

1. **Create and Configure the Database**
2. In the Azure Portal, create an Azure SQL Database.
3. Access the database using **SQL Server Management Studio (SSMS)** or another SQL client by providing the server's username and password.
4. Import your local database to the Azure SQL database using generated SQL scripts or migration tools.
5. **Update the Connection String**
6. Replace your local connection string with the Azure database connection string in your application settings:

```
"DefaultConnection": "Server=<your-server>.database.windows.net;Database=<your-database>;User Id=<your-username>;Password=<your-password>;Encrypt=True;TrustServerCertificate=False;MultipleActiveResultSets=True;Connection Timeout=60;"
```

7. **Create an Azure App Service**
 - a. In the Azure Portal, create a new **App Service**.
 - b. Configure it with **Basic Authentication** and select the appropriate runtime stack (e.g., .NET, Python, Node.js).
8. **Publish the Application**
 - a. Use your development IDE (e.g., Visual Studio, Visual Studio Code) to publish the web application to the App Service.
 - b. Ensure the published files are accessible via the Azure App Service URL.
9. **Testing the Application**
 - a. If your application uses Swagger for API testing, the URL will be `https://<your-app-service-name>.azurewebsites.net/swagger/index.html`

Option 2: Deploying via Containerization Using Docker Hub

Steps to Build and Push Docker Image

1. **Navigate to Container Settings:**
 - a. In the Azure Portal, go to your **App Service**.
 - b. Under **Settings**, click on **Container Settings**.
2. **Set Docker Hub Configuration:**
 - a. **Access Type:**
 - i. Choose **Public** if the Docker Hub repository is public.

- ii. Choose **Private** if the Docker Hub repository is private.
- b. **Registry Server URL:** `https://index.docker.io`
- c. **Username:** Your Docker Hub username.
- d. **Password:** Your Docker Hub password or a Docker Hub Access Token.
- e. **Image and Tag:** Specify the image and tag in the format repository-name:image-tag, e.g., myapp:latest.

3. Deploy Using Deployment Center

- a. Navigate to the **Deployment Center** in the Azure App Service.
- b. Confirm your Docker Hub settings and start the deployment process.

4. Automated Updates

Once configured, any new image pushed to Docker Hub with the specified tag (e.g., latest) will automatically deploy to the Azure App Service.

Below is the screen shot:

Create Web App ...

Basics Database Container Networking Monitor + secure Tags Review + create

Select your preferred source for container images. You can change these settings and other dependencies after creating the app. [Learn more](#)

Sidecar support

☒ Enhanced configuration with sidecar support off [Learn More](#)

Image Source *

☐ Quickstart
☐ Azure Container Registry
☒ Other container registries

Docker hub options

Access Type *

☒ Public
☐ Private

Registry server URL *

Image and tag *

Startup Command ⓘ

Review + create

< Previous

Next : Networking >

In the Deployment center page:

Registry settings

Container type

Single Container

Registry source

Docker Hub

Repository Access *

Public

Full Image Name and Tag *

yunusalirony/backend-app:latest

Startup file or command

Continuous deployment

☐ On ☒ Off

If the Basic authentication is disabled what is needed for deployment check below things.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

yunusali.rony@techjays....
DEFAULT DIRECTORY

Home > Microsoft.Web-WebApp-Portal-cd0a8d30-8d1d | Overview > backendapp-1 | Deployment Center >

Configuration

Refresh Save Discard Leave Feedback

Click here to upgrade to a higher SKU and enable additional features.

View and edit your application settings and connection strings from Environment variables. Click here to go to Environment Variables menu

General settings Path mappings Error pages (preview)

Stack settings

Startup Command

Provide an optional startup command that will be run as part of container startup. [Learn more](#)

Platform settings

SCM Basic Auth Publishi... ☒ On ☐ Off

FTP Basic Auth Publishi... ☐ On ☒ Off

5. Check Application Logs for Listening Port

- After deployment, check the application logs in the Azure Portal to verify the listening port. Navigate to:
 - App Service > Logs > Log Stream
- Look for logs indicating the port where the application is listening (e.g., Application is listening on port 8080).

6. Configure the Correct Listening Port

- If the application is not listening on the expected port, configure the WEBSITES_PORT application setting:
 - Go to App Service > Configuration > Application Settings.
 - Add a new setting:
 - Key: WEBSITES_PORT
 - Value: <listening-port> (e.g., 8080).
 - Save and restart the App Service

test-backend-app | Environment variables

Web App

Search

App settings Connection strings

Search + Add Refresh Show values Advanced edit Pull reference values

Name	Value	Deployment slot setting	Source	Delete
DOCKER_REGISTRY_SERVER_URL	Show value		App Service	Delete
WEBSITES_ENABLE_APP_SERVICE_STORAGE	Show value		App Service	Delete
WEBSITES_PORT	8080		App Service	Delete

Diagnose and solve problems

Microsoft Defender for Cloud

Events (preview)

Recommended services (preview)

Deployment

Deployment slots

Deployment Center

Settings

Environment variables

Configuration

Option 3: Deploying via CI/CD Pipeline Using GitHub Actions

Prerequisites

1. Azure App Service
 - Create an App Service in Azure and note its name and region.
2. GitHub Repository
 - Host your application code in a GitHub repository.
3. Azure Publish Profile
 - Download the App Service's publish profile:
 - Go to your App Service in Azure, click Get Publish Profile, and save the .PublishSettings file.
(If unable to download, navigate to the Deployment Center for assistance.)
4. GitHub Personal Access Token
 - Ensure your GitHub branch has workflow permissions and set a Personal Access Token for authentication if required.

Steps

1. Prepare the GitHub Repository
 - Commit your application code to the repository.
 - Create a .github/workflows folder and add a YAML file (e.g., deploy-to-azure.yml) for the deployment workflow.
2. Create a GitHub Secret for the Publish Profile
 - Go to your GitHub repository:
Settings > Secrets and Variables > Actions > New Repository Secret.
 - Add a secret named AZURE_WEBAPP_PUBLISH_PROFILE.
 - Paste the contents of the .PublishSettings file as the secret value.

Below are the screenshots for the insights

The screenshot shows the 'Source' section of the Azure App Service deployment configuration. It has three radio button options: 'Container Registry', 'GitHub Actions' (which is selected), and 'Azure Pipelines'. Below this is the 'GitHub Actions' section, which contains explanatory text about how App Service will place a workflow in the repository and a 'Learn more' link. Further down are fields for 'Signed in as' (talukderroni13039), 'Organization' (talukderroni13039), 'Repository' (backendapp-source), and 'Branch' (main). At the bottom, the 'Workflow Option' is set to 'Add a workflow'.

Source *

- ☐ Container Registry: Set up your app to pull the container image from a registry.
- ☒ GitHub Actions: Build, deploy, and manage your container app automatically with GitHub Actions.
- ☐ Azure Pipelines: Configure a robust deployment pipeline for your application using Azure Pipelines, part of Azure DevOps Services (formerly known as VSTS).

GitHub Actions

App Service will place a GitHub Actions workflow in your chosen repository to build and deploy your app whenever there is a commit on the chosen branch. If you can't find an organization or repository, you may need to enable additional permissions on GitHub. You must have write access to your chosen GitHub repository to deploy with GitHub Actions.
[Learn more](#)

Signed in as talukderroni13039 [Change Account](#) ⓘ

Organization * talukderroni13039 ▼

Repository * backendapp-source ▼

Branch * main ▼

Workflow Option * ☒ Add a workflow: Add a new workflow file 'main_backendapp-1.yml' in the selected repository and branch.

Registry settings

Registry source

Server URL *

Login

Password

This field is required

Image name *

Tag

Startup file or command

Workflow Configuration

Password will be GitHub personal access token:

Make sure your branch has workflow permissions:

github.com/talukderroni13039/backendapp-source/settings/actions

☒ Require approval for first-time contributors
Only users who have never had a commit or pull request merged into this repository will require approval to run workflows.

☐ Require approval for all external contributors
All users that are not a member or owner of this repository will require approval to run workflows.

Save

Workflow permissions
Choose the default permissions granted to the GITHUB_TOKEN when running workflows in this repository. You can specify more granular permissions in the workflow using YAML. [Learn more about managing permissions.](#)

☒ Read and write permissions
Workflows have read and write permissions in the repository for all scopes.

☐ Read repository contents and packages permissions
Workflows have read permissions in the repository for the contents and packages scopes only.

Choose whether GitHub Actions can create pull requests or submit approving pull request reviews.

☒ Allow GitHub Actions to create and approve pull requests

Save

3. Write the Workflow YAML Example GitHub Actions workflow for deploying a .NET application:

Docs for the Azure Web Apps Deploy action: <https://github.com/Azure/webapps-deploy>

More GitHub Actions for Azure: <https://github.com/Azure/actions>

name: Build and deploy container app to Azure Web App - test-backend-app

on:

push:

branches:

- main

workflow_dispatch:

jobs:

build:

runs-on: ubuntu-latest

steps:

- **uses: actions/checkout@v4**
- **name: Set up Docker Buildx**
uses: docker/setup-buildx-action@v2
- **name: Log in to container registry**
uses: docker/login-action@v2
with:
registry: https://ghcr.io/
username: \${{ github.actor }}
password: \${{ secrets.GITHUB_TOKEN }}
- **name: Build and push container image to registry**
uses: docker/build-push-action@v3
with:
context: ./Backend # Updated: Specify path to Backend folder
file: ./Backend/Dockerfile # Dockerfile location
push: true
tags: ghcr.io/talukderroni13039/backendapp-source:latest

deploy:

runs-on: ubuntu-latest
needs: build
environment:
name: 'Production'
url: \${{ steps.deploy-to-webapp.outputs.webapp-url }}

steps:

- **name: Deploy to Azure Web App**
id: deploy-to-webapp
uses: azure/webapps-deploy@v2
with:
app-name: 'test-backend-app'
publish-profile: \${{ secrets.AZURE_WEBAPP_PUBLISH_PROFILE_TEST }}
images: ghcr.io/talukderroni13039/backendapp-source:latest'

4. Monitor the Workflow

Go to the Actions tab in your GitHub repository to monitor the workflow progress.

Test the Deployment

After the workflow completes, navigate to your App Service URL:

`https://<your-app-service>.azurewebsites.net`

Verify that the application is running as expected.

Conclusion

This document provides three deployment options tailored to various scenarios. Whether you're deploying manually, using containers, or automating through CI/CD pipelines, Azure App Service offers a flexible and robust platform for your web application

➤ <https://github.com/talukderroni13039/backendapp-source>